

Best Practices Report

Guidelines for Effective Collaboration

with AI Coding Agents in Complex Software Projects

Subject: Developer & AI Agent Pair Programming

Context: APICraft API Testing Full-stack Application

Date: July 2026

1. Executive Summary

Agentic AI software engineers are highly capable of navigating multi-file codebases, refactoring features, and automating test procedures. However, without strict governance, AI agents can suffer from context drift, make poor architectural assumptions, and degrade codebase quality. This report compiles the key collaboration best practices derived from our development sessions on the APICraft project.

2. The Two-Phase Workflow (Design Before Code)

The absolute most critical best practice is enforcing a strict boundary between the design phase and the execution phase. An agent must not be permitted to write implementation code until the human developer has approved the blueprint.

- Phase 1: Design & Alignment: Request the agent to produce a structured "implementation_plan.md". This plan must outline the architectural rationale, file changes, data flow, state management, and potential side-effects before any source code is changed.
- Phase 2: Review Gate: The human acts as a gatekeeper. If the plan lacks specificity, reuse of components, or contains bad design patterns, the plan must be rejected. The agent only proceeds upon receiving explicit approval.

3. Architectural Separation (SRP & Utility Isolation)

Keep React components and backend routes focused strictly on presentation and thin coordination. Complex logic must be isolated to make it easier for both agents and humans to debug and maintain.

- Isolate Complexity: Extract data transformation, formatting, and file serialization code (such as `generateCurlCommand`) into pure, vanilla JavaScript/Python modules (e.g., `requestExportUtils.js`).
- Improve Testability: Pure utilities have no dependencies on frameworks, React hooks, or context states, making them 100% testable via isolated unit test scripts.
- Adhere to Component Limits: Enforce a strict line-limit (e.g., ~150 lines of JSX) for UI files. If a component grows beyond this, require the agent to extract logic into helpers or custom hooks.

4. Codebase Governance (AGENTS.md Rules)

Grounding the agent in explicit project instructions prevents style drift, improper library imports, and architectural degradation.

- **Ground Rules File:** Create a permanent AGENTS.md or CLAUDE.md in the workspace customization root containing codebase conventions, style guidelines, and forbidden patterns.
- **State Model Compliance:** Enforce a strict state model (e.g., global auth context, feature-level hooks, and local UI state) to prevent redundant states or prop-drilling.
- **Track Structure Maps:** Maintain a living FRONTEND_STRUCTURE.md and BACKEND_STRUCTURE.md, and require the agent to update them as files are added or modified, keeping codebase maps fresh.

5. Multi-Tier Verification Loops

Testing must be requirement-driven and verified across multiple levels to catch all potential bugs.

- **Tier 1: Backend Integration Tests:** Execute real API calls against test databases using frameworks like pytest. Ensure endpoints are documented in openapi.yaml and verify drift.
- **Tier 2: Component Smoke Tests:** Confirm components render without crashing. Test the pure helper utilities with Jest unit tests covering edge cases (such as single-quote escaping).
- **Tier 3: Browser E2E Verification:** For UI features, use a browser agent to log in, interact with the app, record a demo, and capture screenshots. This catches CSS rendering bugs and routing failures.

6. Summary Checklist for AI Agent Tasks

Collaboration Stage	Deliverable / Action Required	Responsible Entity
1. Clarification	Map options, discuss import/export scopes, align on UX	Developer & Agent
2. Planning	Generate implementation_plan.md detailing all files	Agent (Architect)
3. Approval Gate	Review Design Review and type explicit approval	Developer
4. Coding	Implement features sequentially following dependency order	Agent (Implementer)
5. Testing	Run pytest (backend) and Jest (frontend utils/smoke)	Agent & Test Runner
6. E2E Verification	Start full stack, verify UI flow, output demo and logs	Agent (Browser)
7. Review Report	Generate review_report.md auditing SRP and coupling	Agent (Reviewer)

Following these checklists keeps development cycles clean, eliminates regressions, and ensures that the developer stays in full control of the codebase architecture.